

Diseño y Ejecución de Pipelines de Integración Continua

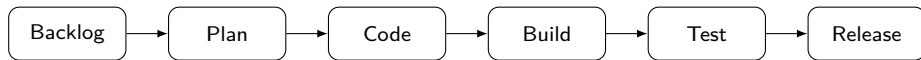
Ingeniería Web Avanzada

Sandra Cano

Pontificia Universidad Católica de Valparaíso

Segundo semestre 2026

¿Dónde se ubica CI dentro de DevOps?



Backlog

Lista priorizada y dinámica que puede contener **requerimientos, funcionalidades, errores, mejoras técnicas y tareas operacionales**.

Relación con esta sesión

Los elementos seleccionados del backlog se desarrollan y producen cambios de código. La **Integración Continua** automatiza la validación de esos cambios mediante build, pruebas y otras verificaciones.

Del requerimiento al Pipeline CI

Ejemplo

Requerimiento: El sistema debe permitir consultar productos por categoría.

Requerimiento → Backlog → Desarrollo → Commit / Pull Request → Pipeline CI

Elementos del Backlog

- Crear filtro de productos en Angular.
- Crear endpoint en NestJS.
- Agregar pruebas del frontend.
- Agregar pruebas del backend.

Estructura de un ítem del Backlog

Backlog

Lista priorizada y dinámica de trabajo pendiente. Puede contener requerimientos, funcionalidades, errores, mejoras técnicas, seguridad, infraestructura o tareas DevOps.

Campo	Descripción
ID	Identificador del ítem. Ejemplo: PB-021.
Título	Nombre breve del trabajo a realizar.
Tipo	Funcionalidad, bug, seguridad, deuda técnica, CI/CD.
Descripción	Qué se necesita y por qué.
Prioridad	Importancia relativa del ítem.
Criterios de aceptación	Condiciones para considerar el ítem terminado.
Estado	Pendiente, en desarrollo, en revisión o terminado.

PB-021: Filtrar productos por categoría

Tipo	Funcionalidad
Prioridad	Alta
Descripción	El usuario debe poder seleccionar una categoría y visualizar sólo los productos asociados.
Criterios	● El usuario puede seleccionar una categoría. ● La lista se actualiza correctamente. ● Si no hay productos, se muestra un mensaje.
Dependencia	Endpoint GET <code>/productos?categoria=</code> en NestJS.

Ejemplo de tarea DevOps en el Backlog

DEVOPS-012: Automatizar validación con CI

Tipo	DevOps / CI
Prioridad	Alta
Descripción	Configurar un workflow de GitHub Actions que valide automáticamente frontend Angular y backend NestJS ante cada Pull Request.
Criterios de aceptación	● Ejecuta <code>npm ci</code> en frontend y backend. ● Ejecuta pruebas automatizadas. ● Ejecuta <code>npm run build</code>. ● El pipeline informa éxito o fallo.
Estado	Pendiente

Idea clave

El backlog no contiene sólo funcionalidades para el usuario; también puede incluir trabajo técnico necesario para **automatizar, desplegar, monitorear o proteger** el sistema.

¿Qué es Angular?

Angular

Framework para construir el **frontend** de aplicaciones web.

Conceptos que necesitamos conocer

- **Componentes:** representan partes de la interfaz.
- **Templates:** definen el HTML que presenta cada componente.
- **Servicios:** encapsulan funcionalidad reutilizable y acceso a datos.
- **Router:** controla la navegación entre vistas.
- **HttpClient:** permite realizar solicitudes HTTP al backend.
- **Inyección de dependencias:** permite utilizar servicios sin crearlos directamente dentro de los componentes.

Ejemplo: frontend Angular



- El **componente** muestra productos.
- El **servicio** encapsula el acceso a datos.
- **HttpClient** realiza la solicitud.
- El backend responde mediante una API.

Ejemplo

GET /api/productos

¿Qué es NestJS?

NestJS

Framework para construir el **backend** de aplicaciones utilizando Node.js y TypeScript.

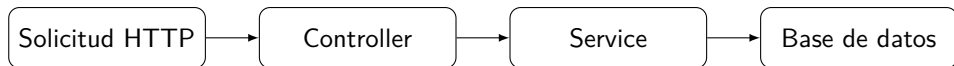
Conceptos básicos

- **Controller:** recibe solicitudes HTTP y entrega respuestas.
- **Service / Provider:** contiene lógica de aplicación reutilizable.
- **Module:** organiza funcionalidades relacionadas.
- **DTO:** representa y valida datos intercambiados.
- **Dependency Injection:** administra las dependencias entre componentes.

Responsabilidad principal

NestJS recibe las solicitudes del frontend, ejecuta la lógica de aplicación y coordina el acceso a datos u otros servicios.

Organización básica de NestJS

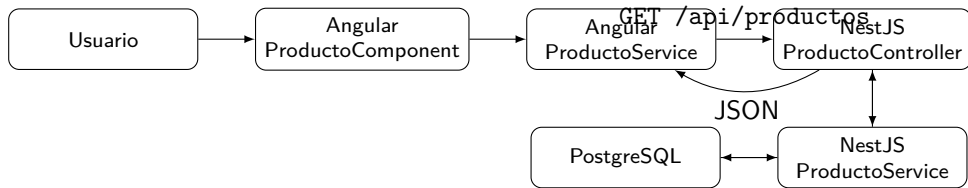


Ejemplo

GET /productos

- 1 El Controller recibe la solicitud.
- 2 Delega la operación al Service.
- 3 El Service obtiene los datos.
- 4 El Controller devuelve la respuesta.

¿Cómo se comunican Angular y NestJS?



Separación de responsabilidades

Angular: interfaz e interacción.

NestJS: API, reglas de aplicación y acceso a datos.

Estructura del Proyecto

```
proyecto/  
|  
|-- frontend/      Angular  
|  
|-- backend/       NestJS  
|  
|-- .github/  
|   |-- workflows/  
|       |-- ci.yml
```

Cada componente posee:

- dependencias propias;
- pruebas propias;
- proceso de build propio;
- posibles errores independientes.

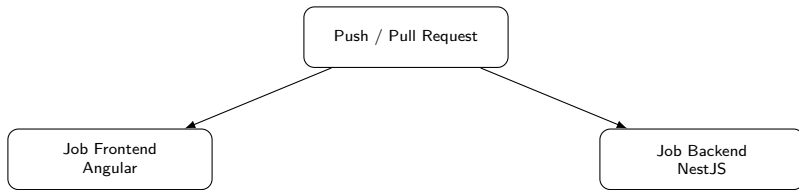
Idea clave

Frontend y backend pueden tener procesos de build y pruebas independientes dentro del mismo pipeline de CI.

Del proyecto al Pipeline CI

Punto de partida

El sistema posee dos componentes que pueden **construirse y probarse de forma independiente**.



Job Frontend

- instalar dependencias;
- ejecutar pruebas;
- construir Angular.

Job Backend

- instalar dependencias;
- ejecutar pruebas;
- construir NestJS.

Concepto	Descripción
Pipeline	Flujo automatizado que valida cambios del repositorio.
Job	Unidad de trabajo que ejecuta comandos específicos.
Stage	Agrupación lógica de jobs que permite ordenar el pipeline.
Step	Acción o comando dentro de un job.
Runner	Máquina o entorno que ejecuta los jobs.
Caché	Archivos reutilizables para acelerar ejecuciones posteriores.
Artefacto	Archivo producido por un job y conservado como resultado del pipeline.

Precisión

La terminología exacta cambia según la plataforma: GitHub Actions habla de workflow, jobs y steps; GitLab CI/CD usa pipeline, stages, jobs y runners.

Comandos básicos dentro del pipeline

Comando	Función
<code>npm ci</code>	Realiza una instalación limpia y reproducible de las dependencias utilizando las versiones registradas en <code>package-lock.json</code> .
<code>npm test</code>	Ejecuta el script <code>test</code> definido en <code>package.json</code> . Normalmente permite ejecutar las pruebas automatizadas del proyecto.
<code>npm run build</code>	Ejecuta el script <code>build</code> definido en <code>package.json</code> y genera una versión compilada o preparada de la aplicación.

Secuencia típica en CI

`npm ci → npm test → npm run build`

¿Qué ocurre en Angular y NestJS?

Frontend Angular

`npm ci`

- instala Angular y dependencias;
- utiliza `package-lock.json`.

`npm test`

- ejecuta las pruebas configuradas.

`npm run build`

- construye la aplicación Angular;
- genera archivos para distribución.

Backend NestJS

`npm ci`

- instala NestJS y dependencias;
- utiliza `package-lock.json`.

`npm test`

- ejecuta las pruebas del backend.

`npm run build`

- compila TypeScript;
- genera la aplicación backend.

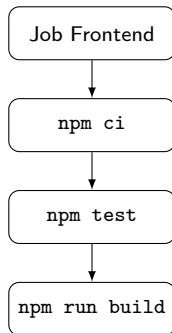
Job y Step: ¿cuál es la diferencia?

Job

Una **unidad de trabajo** del pipeline que se ejecuta en un runner determinado.

Steps

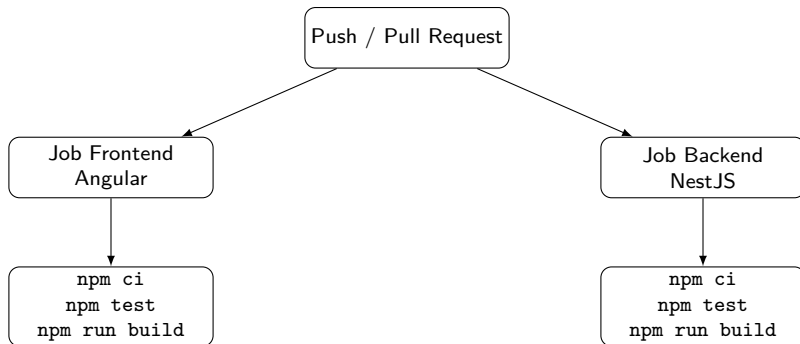
Son las **acciones o comandos secuenciales** que se ejecutan dentro de un job.



Paralelismo en un pipeline

Idea principal

Cuando dos jobs no dependen entre sí, pueden ejecutarse **en paralelo** para reducir el tiempo total del pipeline.



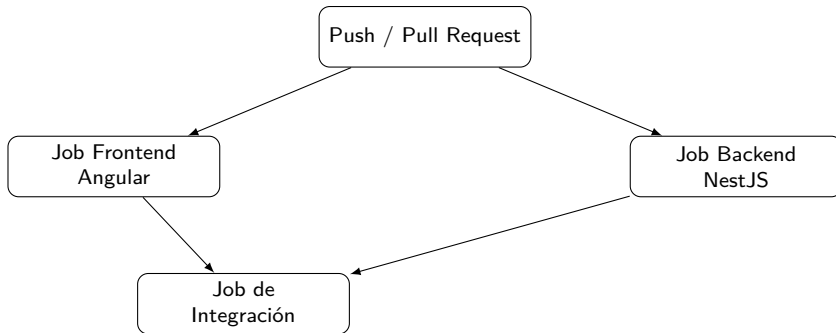
Ejemplo

El frontend Angular y el backend NestJS pueden validarse al mismo tiempo porque sus pruebas iniciales son independientes.

Dependencias entre jobs

Idea principal

No todos los jobs pueden ejecutarse de forma independiente. Algunos necesitan que otros jobs finalicen correctamente antes de comenzar.



Ejemplo

El job de integración sólo puede comenzar cuando **frontend y backend han finalizado correctamente**.

Dependencias con needs

```
jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - run: npm ci
      - run: npm test
      - run: npm run build
  backend:
    runs-on: ubuntu-latest
    steps:
      - run: npm ci
      - run: npm test
      - run: npm run build
  integration:
    needs:
      - frontend
      - backend
    runs-on: ubuntu-latest
    steps:
      - run: npm run test:integration
```

GitHub Actions: Frontend Angular + Backend NestJS

```
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - name: Obtener codigo
        uses: actions/checkout@v6
      - name: Configurar Node.js
        uses: actions/setup-node@v7
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json
      - name: Instalar dependencias
        working-directory: frontend
        run: npm ci
      - name: Ejecutar pruebas
        working-directory: frontend
        run: npm test
      - name: Construir Angular
        working-directory: frontend
        run: npm run build
```

GitHub Actions: Frontend Angular + Backend NestJS

```
backend:
  runs-on: ubuntu-latest
  steps:
    - name: Obtener codigo
      uses: actions/checkout@v6
    - name: Configurar Node.js
      uses: actions/setup-node@v7
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: backend/package-lock.json
    - name: Instalar dependencias
      working-directory: backend
      run: npm ci
    - name: Ejecutar pruebas
      working-directory: backend
      run: npm test
    - name: Construir NestJS
      working-directory: backend
      run: npm run build
```

Runners

¿Qué es un runner?

Un **runner** es el entorno que ejecuta los jobs del pipeline. Puede ser una máquina administrada por la plataforma o una máquina propia del equipo.

Runner administrado

- Disponible bajo demanda.
- Menor configuración inicial.
- Entorno limpio por ejecución.
- Ejemplo: Ubuntu alojado por GitHub/GitLab.

Runner propio

- Control sobre hardware y red.
- Útil para entornos privados.
- Requiere administración y seguridad.
- Ejemplo: servidor institucional.

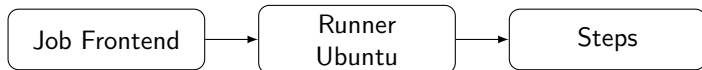
Pregunta

¿Qué riesgo aparece si el runner propio tiene credenciales o acceso a redes internas?

Runner: ¿dónde se ejecuta un Job?

Runner

Un **runner** es la máquina o entorno encargado de ejecutar los **steps** definidos dentro de un job.



- El runner prepara el entorno de ejecución.
- Ejecuta los comandos del job.
- Diferentes jobs pueden usar runners distintos.
- Los runners pueden ser administrados por GitHub o por la propia organización.

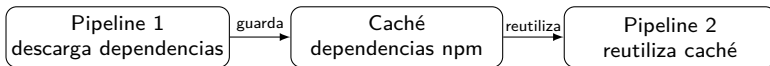
Ejemplo

`runs-on: ubuntu-latest` indica que el job se ejecutará en un runner con Ubuntu.

Caché: acelerar la ejecución del pipeline

¿Qué es la caché?

La **caché** permite conservar y reutilizar archivos entre distintas ejecuciones del pipeline para evitar operaciones repetitivas y reducir el tiempo de ejecución.



Objetivo

La caché busca **mejorar el rendimiento del pipeline**; no representa el producto final generado por el build.

Ejemplo: caché de dependencias npm

```
- name: Configurar Node.js
  uses: actions/setup-node@v7
  with:
    node-version: '20'
    cache: 'npm'
    cache-dependency-path: frontend/package-lock.json

- name: Instalar dependencias
  working-directory: frontend
  run: npm ci
```

¿Qué ocurre?

- 1 El runner busca una caché asociada a las dependencias.
- 2 Si existe, reutiliza los archivos descargados previamente.
- 3 npm ci sigue verificando e instalando las dependencias definidas en el lockfile.

Artefactos

¿Qué es un artefacto?

Un **artefacto** es un archivo o conjunto de archivos producido por un job y conservado como resultado del pipeline.

Ejemplos de artefactos:

- carpeta `dist/` de Angular;
- binarios o archivos comprimidos;
- reporte de pruebas;
- cobertura de código;
- logs o evidencias de ejecución.

Pregunta

Si se genera `dist/frontend`, ¿conviene guardarlo como caché o como artefacto?

Caché vs artefactos

Criterio	Caché	Artefacto
Objetivo	Acelerar ejecuciones	Guardar resultados
Ejemplo	Dependencias npm	dist/, reportes, cobertura
Vida útil	Puede reutilizarse entre pipelines	Resultado de un pipeline o job
Debe ser reproducible	Sí	No necesariamente
Uso típico	/.npm, .gradle	build, reporte de tests

Regla práctica

Use caché para dependencias reutilizables; use artefactos para resultados que quiere conservar, revisar o pasar a otro job.

Pipeline CI: Angular + NestJS (1/2)

```
name: CI Angular + NestJS
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
permissions:
  contents: read
jobs:
  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - name: Obtener codigo
        uses: actions/checkout@v7
      - name: Configurar Node.js
        uses: actions/setup-node@v7
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json
      - name: Instalar dependencias
        run: npm ci
      - name: Ejecutar pruebas
        run: npm test
      - name: Construir Angular
        run: npm run build
      - name: Guardar build frontend
        uses: actions/upload-artifact@v7
        with:
          name: frontend-dist
          path: frontend/dist/
```

Pipeline CI: Angular + NestJS (2/2)

```
backend:
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: backend
  steps:
    - name: Obtener codigo
      uses: actions/checkout@v7
    - name: Configurar Node.js
      uses: actions/setup-node@v7
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: backend/package-lock.json
    - name: Instalar dependencias
      run: npm ci
    - name: Ejecutar pruebas
      run: npm test
    - name: Construir NestJS
      run: npm run build
    - name: Guardar build backend
      uses: actions/upload-artifact@v7
      with:
        name: backend-dist
        path: backend/dist/
  integration:
    needs: [frontend, backend]
    runs-on: ubuntu-latest
    steps:
      - name: Obtener codigo
        uses: actions/checkout@v7
      - name: Ejecutar pruebas de integracion
        run: npm run test:integration
```

Ejemplo equivalente en GitLab CI/CD

```
stages:
  - install
  - test
  - build

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - frontend/node_modules/

frontend-test:
  stage: test
  image: node:20
  script:
    - cd frontend
    - npm ci
    - npm test --if-present

frontend-build:
  stage: build
  image: node:20
  script:
    - cd frontend
    - npm ci
    - npm run build

artifacts:
  paths:
    - frontend/dist/
  expire_in: 1 week
```

- 1 ¿Qué evento activa el pipeline?
- 2 ¿Cuántos jobs existen?
- 3 ¿Qué runner utiliza cada job?
- 4 ¿Qué comandos corresponden a pruebas?
- 5 ¿Dónde aparece la caché?
- 6 ¿Qué archivos podrían guardarse como artefactos?

Pregunta	Respuesta esperada
Evento	push o pull_request.
Jobs	Uno o más trabajos independientes, por ejemplo frontend y backend.
Runner	ubuntu-latest o imagen Docker en GitLab.
Pruebas	npm test, npm run test, pytest, etc.
Caché	Dependencias descargadas: /.npm, node_modules, .gradle.
Artefactos	dist/, reportes de pruebas, cobertura, logs.

Actividad: decidir caché o artefacto

Instrucción

Clasifique cada elemento como **caché**, **artefacto** o **ninguno**.

Elemento	Clasificación
Dependencias descargadas por npm	
Carpeta dist/ del frontend	
Archivo coverage/lcov.info	
Contraseña de base de datos	
Log de pruebas fallidas	
package-lock.json	

Solución: caché o artefacto

Elemento	Clasificación	Justificación
Dependencias npm	Caché	Aceleran ejecuciones posteriores.
dist/	Artefacto	Es resultado del build.
Cobertura	Artefacto	Evidencia de pruebas.
Contraseña BD	Ninguno	Nunca debe guardarse en caché ni artefactos.
Log de pruebas	Artefacto	Sirve para diagnóstico.
package-lock.json	Repositorio	Debe versionarse, no generarse como artefacto.

Buenas prácticas en CI

- Mantener el pipeline rápido: ejecutar primero validaciones simples.
- Usar `npm ci` cuando existe `package-lock.json`.
- Separar jobs cuando frontend y backend tienen ciclos distintos.
- Usar caché para dependencias, no para resultados finales.
- Guardar artefactos relevantes: build, logs, cobertura.
- No guardar secretos en el repositorio, caché ni artefactos.
- Hacer que el pipeline falle cuando una validación crítica falla.
- Documentar en el README cómo ejecutar localmente lo que hace CI.

Idea clave

Un buen pipeline CI debe ser reproducible, comprensible y útil para tomar decisiones.

Error	Consecuencia
No ejecutar pruebas en Pull Request	Se integran fallos al branch principal.
Usar caché como si fuera artefacto	Se pierden resultados o se reutilizan archivos incorrectos.
Guardar secretos en variables visibles	Riesgo de exposición de credenciales.
Pipeline demasiado lento	El equipo evita usarlo o ignora sus resultados.
No fijar versión de Node.js	Resultados distintos entre máquinas.
No separar frontend/backend	Falla difícil de diagnosticar en monorepos.

Caso

Un equipo desarrolla un sistema con Angular y NestJS. El frontend cambia con frecuencia, pero el backend tiene pruebas más lentas. El pipeline actual ejecuta todo en un único job de 12 minutos.

Preguntas:

- 1 ¿Qué problema tiene este pipeline?
- 2 ¿Qué jobs separaría?
- 3 ¿Qué usaría como caché?
- 4 ¿Qué guardaría como artefacto?
- 5 ¿Qué validación debería bloquear un Pull Request?

❶ Problema del pipeline

Un único job ejecuta todas las tareas de forma secuencial. Esto aumenta el tiempo total y no aprovecha que frontend y backend pueden validarse de manera independiente.

❷ Jobs a separar

- frontend: instalar, probar y construir Angular.
- backend: instalar, probar y construir NestJS.
- integration: validar la comunicación entre ambos.

❸ Caché

Dependencias descargadas por npm, utilizando como referencia los archivos `package-lock.json` de frontend y backend.

❹ Artefactos

- build de Angular: `frontend/dist/`;
- build de NestJS: `backend/dist/`;
- opcionalmente, reportes de pruebas o cobertura.

❺ Validación que bloquea el Pull Request

El Pull Request no debería integrarse si fallan las pruebas, el build o las validaciones de integración.

Ticket de salida

- 1 Defina **pipeline** con sus propias palabras.
- 2 Explique la diferencia entre **job** y **runner**.
- 3 ¿Qué diferencia existe entre **caché** y **artefacto**?
- 4 Escriba una validación que debería ejecutarse en CI para un proyecto Angular.
- 5 Escriba una validación que debería ejecutarse en CI para un backend NestJS.

Cinco ideas clave

- 1 CI valida cambios automáticamente antes de integrarlos.
- 2 Un pipeline organiza jobs, stages y steps.
- 3 Los runners son los entornos que ejecutan los jobs.
- 4 La caché acelera el pipeline; los artefactos conservan resultados.
- 5 CI es una práctica técnica y también una forma de trabajo colaborativo.

Conexión con DevOps

CI es el primer paso para avanzar hacia entrega continua, despliegue continuo y operación confiable del software.

- GitHub Docs. *Workflow syntax for GitHub Actions*. Disponible en la documentación oficial de GitHub.
- GitHub Docs. *Dependency caching* y *Workflow artifacts*. Disponible en la documentación oficial de GitHub Actions.
- GitHub Docs. *Building and testing Node.js*. Disponible en la documentación oficial de GitHub Actions.
- GitLab Docs. *CI/CD pipelines*, *CI/CD jobs* y *Caching in GitLab CI/CD*. Disponible en la documentación oficial de GitLab.